

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)



SBT-DF203 LAB 1 REPORT **HTTP Analysis Using Wireshark: Text Traffic**

Student Name: Abubakari-Sadique Hamidu

Student ID: 2025-FWSD-11392

Basic Networking Skills for Digital Forensics
TCP Handshake Reconstruction, HTTP Request/Response Analysis,
and Network Forensic Evidence Reporting

Lab Title: HTTP Analysis Using Wireshark Text Traffic

Instructor: Aminu Idris (CCNA, CompTIA, CEH, OSCP, CISSP, CISM)

Case/Lab Identifier: SBT-DF203-Lab1-AbubakariSadiqueHamidu

Date: 8 September 2026

Tools and Resources Used

- **Docker Desktop**: used to run an isolated Kali Linux container on my own laptop instead of a full virtual machine.
- **Kali Linux** (kalilinux/kali-rolling image): the operating system everything ran on.
- **Apache2, curl, and tshark**—the actual tools used to serve the page, request it, and record the traffic.
- **Git and GitHub**: used to save and submit the evidence, screenshots, and this report.
- **Google**: used a few times to look up specific error messages I hit along the way, like the Wireshark install failure.
- **Claude and Google**: used throughout the lab to help troubleshoot problems as they came up (like the failed capture attempt and the Docker group error).

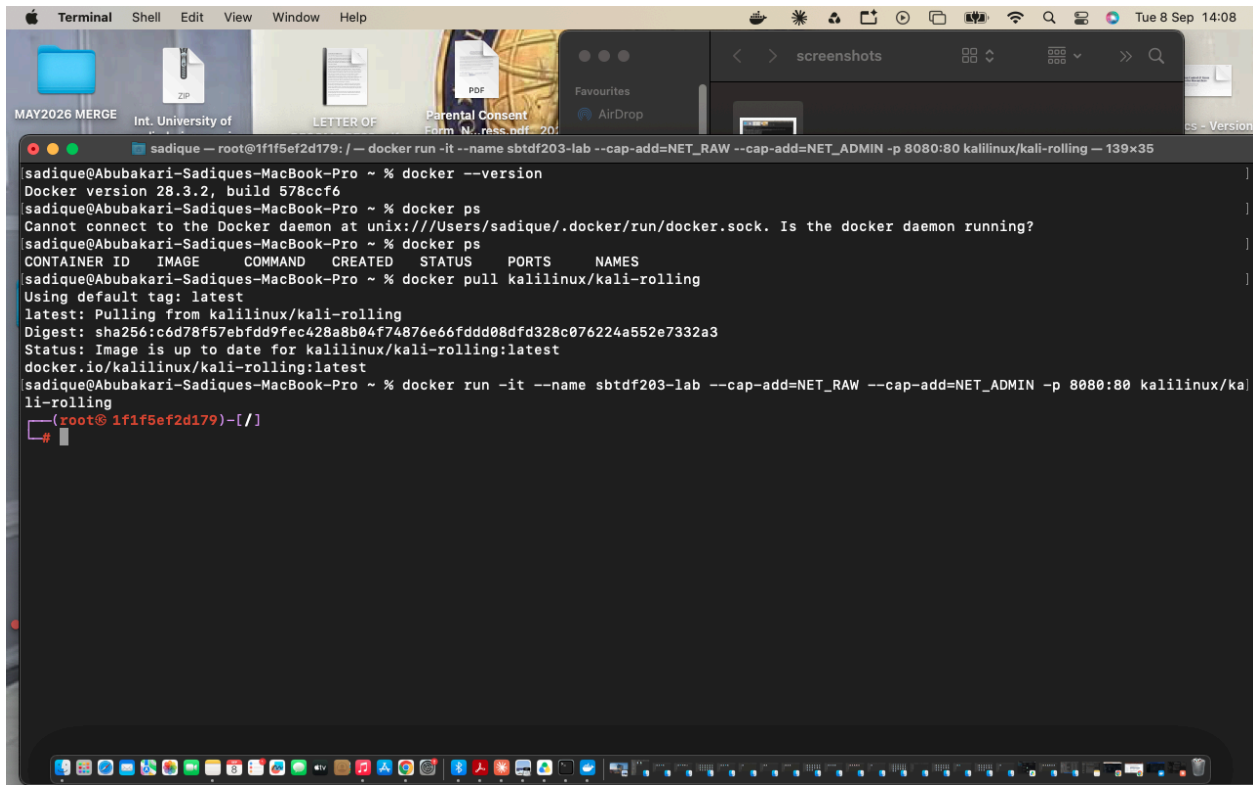
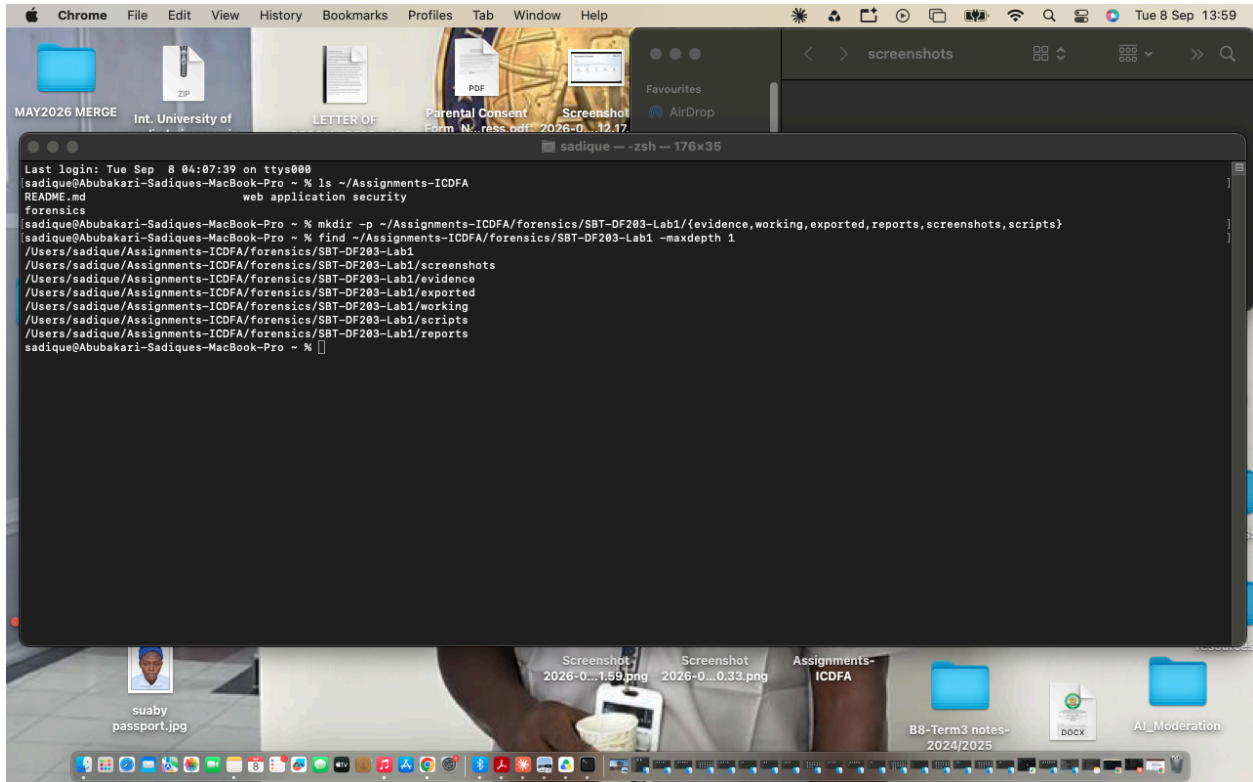
Executive Summary

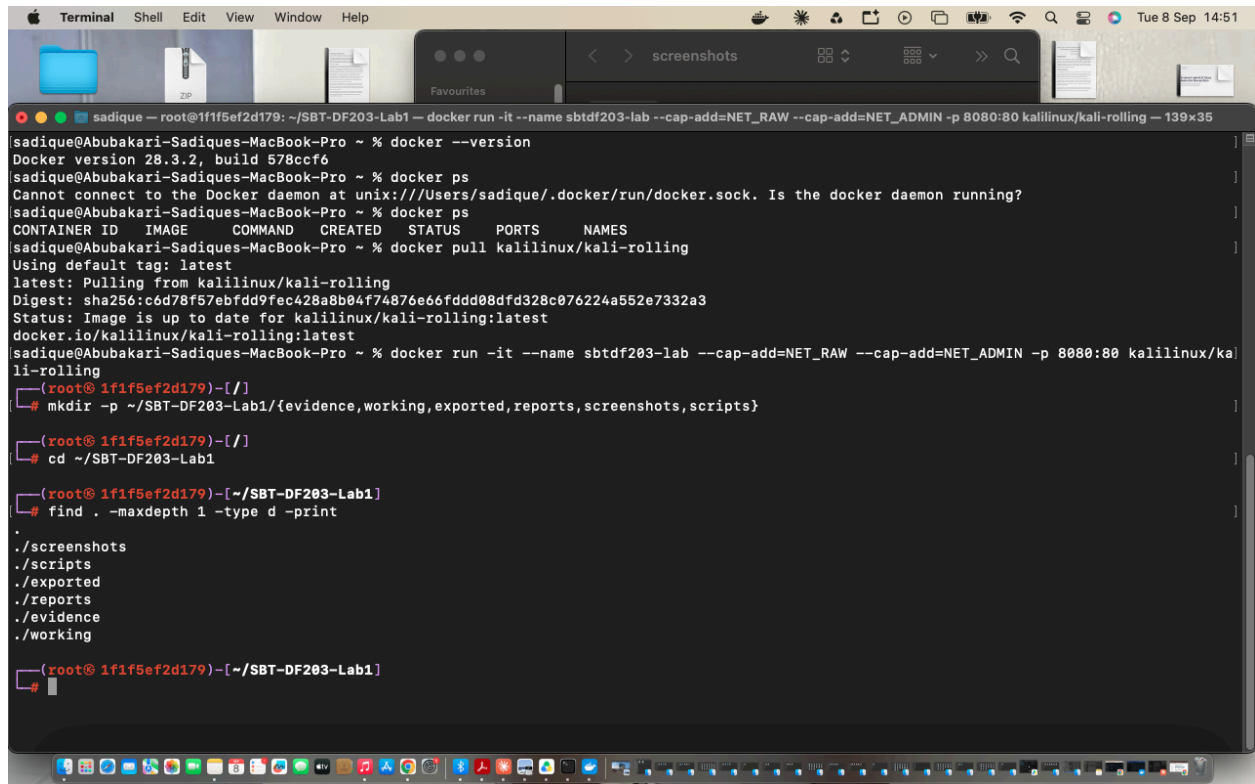
For this lab, I built a small, private webpage, watched the exact moment a computer requests that page and gets a response, and explained what happens at each stage. I did all of it inside an isolated Kali Linux container on my own laptop; nothing here ever touched a real network, a real website, or anyone else's computer.

This report is written in the order I actually did the work, step by step, using plain words instead of technical jargon wherever I can. Where I do need to use a technical word, I explain what it means the first time it shows up.

Step 1: Setting Up a Workspace

Before touching any tools, I made a set of folders to keep everything tidy: one for the original recording of the network traffic, one for a safe copy I could actually work on, one for reports, and one for screenshots. Keeping the original separate from the copy matters; if I make a mistake while poking around later, I've still got the untouched original to fall back on.



A screenshot of a macOS Terminal window. The title bar shows 'Terminal' with menu options 'Shell', 'Edit', 'View', 'Window', and 'Help'. The status bar at the top right indicates 'Tue 8 Sep 14:51'. The terminal content shows a user named 'sadique' running Docker commands. First, they check the Docker version (28.3.2). Then, they attempt to run 'docker ps' but get an error: 'Cannot connect to the Docker daemon at unix:///Users/sadique/.docker/run/docker.sock. Is the docker daemon running?'. Next, they pull the 'kalilinux/kali-rolling' image. Finally, they run 'docker run' with various flags to start a container named 'sbtDF203-lab'. Inside the container, they run 'mkdir -p ~/SBT-DF203-Lab1/{evidence,working,exported,reports,screenshots,scripts}' and 'cd ~/SBT-DF203-Lab1'. Then, they run 'find . -maxdepth 1 -type d -print' which lists several directories: './screenshots', './scripts', './exported', './reports', './evidence', and './working'.

```
sadique -- root@1f1f5ef2d179: ~/SBT-DF203-Lab1 -- docker run -it --name sbtDF203-lab --cap-add=NET_RAW --cap-add=NET_ADMIN -p 8080:80 kalilinux/kali-rolling -- 139x35
sadique@Abubakari-Sadiques-MacBook-Pro ~ % docker --version
Docker version 28.3.2, build 578ccf6
sadique@Abubakari-Sadiques-MacBook-Pro ~ % docker ps
Cannot connect to the Docker daemon at unix:///Users/sadique/.docker/run/docker.sock. Is the docker daemon running?
sadique@Abubakari-Sadiques-MacBook-Pro ~ % docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS        NAMES
sadique@Abubakari-Sadiques-MacBook-Pro ~ % docker pull kalilinux/kali-rolling
Using default tag: latest
latest: Pulling from kalilinux/kali-rolling
Digest: sha256:c6d78f57ebfdd9fec428a8b04f74876e66fddd08dfd328c076224a552e7332a3
Status: Image is up to date for kalilinux/kali-rolling:latest
docker.io/kalilinux/kali-rolling:latest
sadique@Abubakari-Sadiques-MacBook-Pro ~ % docker run -it --name sbtDF203-lab --cap-add=NET_RAW --cap-add=NET_ADMIN -p 8080:80 kalilinux/kali-rolling
(root@ 1f1f5ef2d179)-[/]
# mkdir -p ~/SBT-DF203-Lab1/{evidence,working,exported,reports,screenshots,scripts}
(root@ 1f1f5ef2d179)-[/]
# cd ~/SBT-DF203-Lab1
(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab1]
# find . -maxdepth 1 -type d -print
.
./screenshots
./scripts
./exported
./reports
./evidence
./working
(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab1]
#
```

Step 2: Installing the Tools

I installed three things: Apache (a program that serves a webpage to anyone who asks for it), curl (a small tool that can ask a webpage for its content and show the reply), and tshark (a tool that records and reads network traffic as it happens).

One thing went wrong here that's worth mentioning honestly: Wireshark's installer needs a user group called "wireshark" to already exist on the system, and in this minimal container it didn't. The install failed with an error explaining exactly that. I fixed it by creating the group myself with one command, and the install finished properly straight after.

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker run -it --name sbtdf203-lab --cap-add=NET_RAW --cap-add=NET_ADMIN -p 8080:80 kali/kali-rolling — 139x35
bash: sudo: command not found

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# apt update
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [21.5 MB]
Get:3 http://kali.download/kali kali-rolling/contrib amd64 Packages [113 kB]
Get:4 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [16.0 kB]
Get:5 http://kali.download/kali kali-rolling/non-free amd64 Packages [183 kB]
Fetched 21.8 MB in 20s (1076 kB/s)
3 packages can be upgraded. Run 'apt list --upgradable' to see them.

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# apt install -y apache2 curl wireshark tshark
Installing:
  apache2  curl  tshark  wireshark

Installing dependencies:
adwaita-icon-theme      libde265-0      libkrb5-3      libre2-11      libxau6
alsa-topology-conf      libdeflate0     libkrb5support libreadline8t64 libxcb-cursor0
alsa-ucm-conf           libdevmapper1.02.1 liblcms2-2     librist4       libxcb-dri3-0
apache2-bin             libdisplay-info3 libldap-common librsvg2-2     libxcb-glx0
apache2-data            libdnnl3.6      libldap2       librsvg2-common libxcb-icccm4
apache2-utils           libdouble-conversion3 liblerc4       libsasl2-2     libxcb-image0
at-spi2-common          libdrm-amdgpu1  liblilv-0-0    libsasl2-modules libxcb-keysyms1
at-spi2-core            libdrm-common   libllvm21      libsasl2-modules-db libxcb-present0
bash-completion         libdrm-intel1   liblua5.4-0    libsbcl         libxcb-randr0
bubblewrap              libdrm2         libmaxminddb0  libselectors-config libxcb-render-util0
ca-certificates         libduktape207   libmd4c0       libsensors5     libxcb-render0
dbus                    libdvdrnav4     libminizip1t64 libserd-0-0     libxcb-shape0
dbus-bin                libdvdrread8t64 libmp3lame0    libsharpvuv0   libxcb-shm0
dbus-daemon             libebur128-1    libmpeg123-0t64 libshine3       libxcb-sync1
dbus-session-bus-common libedit2         libmtdev1t64   libsm6          libxcb-util1
dbus-system-bus-common libegl-mesa0     libmysofa1     libsmi2t64      libxcb-xfixes0
dbus-user-session       libegl1         libncursesw6    libsnappy1v5    libxcb-xinput0
```

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker run -it --name sbtdf203-lab --cap-add=NET_RAW --cap-add=NET_ADMIN -p 8080:80 kali/kali-rolling — 139x35
bash: sudo: command not found

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# service apache2 start
Starting Apache httpd web server: apache2.

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# service apache2 status
apache2 is running.

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# ss -lntp | grep ':80'
LISTEN 0      511          0.0.0.0:*      0.0.0.0:*      users:((("apache2",pid=7627,fd=3))

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
#
```

Step 3 Building the Webpage

I made a very simple webpage with just my name and my student ID written on it and had Apache serve it at address 127.0.0.1 (this just means "this same computer"), on port 80. Port 80 is simply the standard "door" that web servers listen at.

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker run -it --name sbtdf203-lab --cap-add=NET_RAW --cap-add=NET_ADMIN -p 8080:80 kali/kali-rolling — 139x35

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# nano /var/www/html/basic.html
bash: nano: command not found

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# vi /var/www/html/basic.html
bash: vi: command not found

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: ABUBAKARI-SADIQUE HAMIDU</p><p>Reg No: 2025-FWSD-11392</p></body></html>\n' | tee /var/www/html/basic.htm
```

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker run -it --name sbtdf203-lab --cap-add=NET_RAW --cap-add=NET_ADMIN -p 8080:80 kali/kali-rolling — 139x35

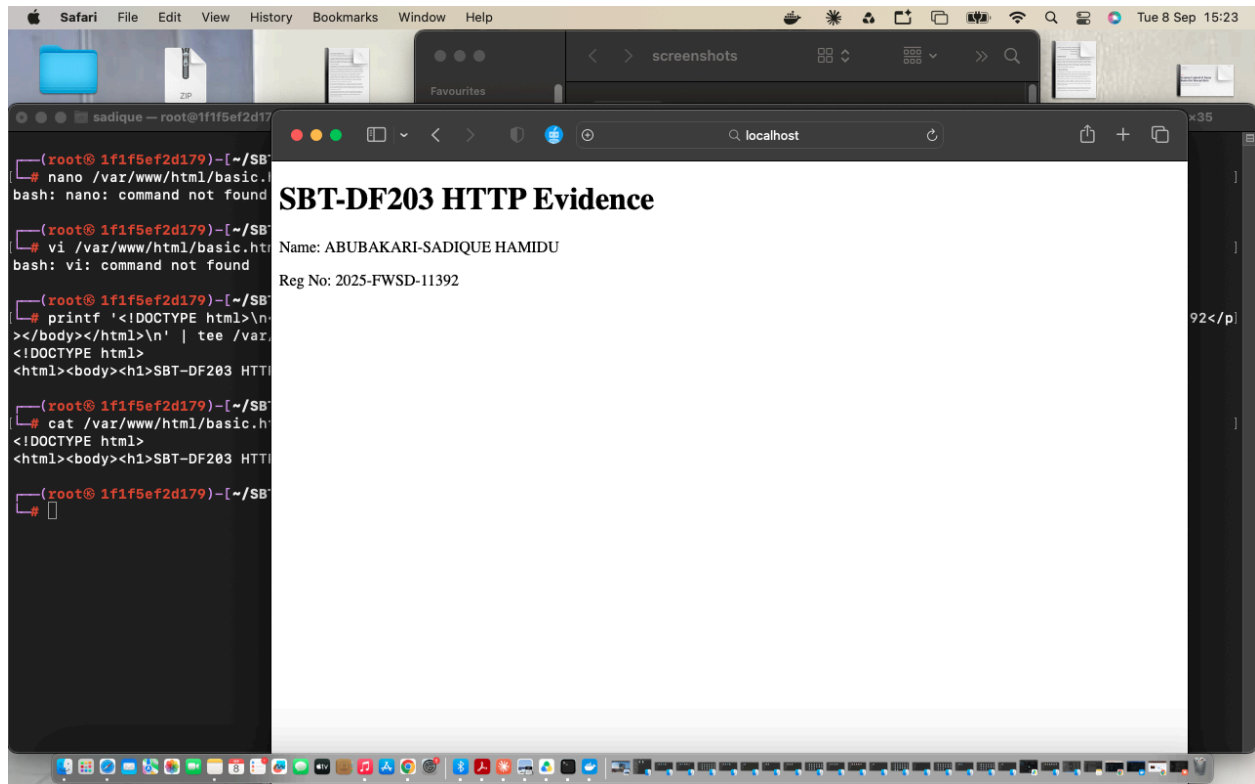
(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# nano /var/www/html/basic.html
bash: nano: command not found

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# vi /var/www/html/basic.html
bash: vi: command not found

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# printf '<!DOCTYPE html>\n<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: ABUBAKARI-SADIQUE HAMIDU</p><p>Reg No: 2025-FWSD-11392</p></body></html>\n' | tee /var/www/html/basic.html
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: ABUBAKARI-SADIQUE HAMIDU</p><p>Reg No: 2025-FWSD-11392</p></body></html>

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# cat /var/www/html/basic.html
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: ABUBAKARI-SADIQUE HAMIDU</p><p>Reg No: 2025-FWSD-11392</p></body></html>

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
#
```



Step 4: Checking the Page Works

Before recording anything, I used curl to ask for the page and read the reply. It came back looking exactly like a normal web reply should: a line saying "200 OK" (which just means "here's your page, no problem"), followed by my name and student ID.

Step 5: Recording the Traffic

Next, I started tshark, which quietly records everything going in and out over the connection. While it was running, I opened a second terminal window and asked for the page again with curl. Once curl got its reply, I stopped the recording.

My first attempt at this didn't actually work. I typed the curl command into the same window tshark was already running in, so nothing got sent while it was recording, and I ended up with an empty file. On the second try, using two separate windows properly, it worked: the recording captured 10 small pieces of data (called packets) in well under one hundredth of a second.


```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker run -it --name sbtdf203-lab --cap...

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
10

sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker exec -it sbtdf203-lab bash

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# curl --no-keepalive -v http://127.0.0.1/basic.html
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 45318
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 08 Sep 2026 16:10:30 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 08 Sep 2026 15:21:57 GMT
< ETag: "8e-65afa4ba94f11"
< Accept-Ranges: bytes
< Content-Length: 142
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: ABUBAKARI-SADIQUE HAMIDU</p><p>WSD-11392</p></body></html>
* Connection #0 to host 127.0.0.1:80 left intact

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
#
```

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker run -it --name sbtdf203-lab --cap...

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
10 ^C

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# tshark -r evidence/basic.pcapng | wc -l
Running as user "root" and group "root". This could be dangerous.
10

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# ls -lh evidence/basic.pcapng
-rw-r----- 1 root root 1.9K Sep  8 16:12 evidence/basic.pcapng

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# tshark -r evidence/basic.pcapng | wc -l
Running as user "root" and group "root". This could be dangerous.
10

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
#

sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker exec -it sbtdf203-lab bash

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# curl --no-keepalive -v http://127.0.0.1/basic.html
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1 port 45318
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 08 Sep 2026 16:10:30 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 08 Sep 2026 15:21:57 GMT
< ETag: "8e-65afa4ba94f11"
< Accept-Ranges: bytes
< Content-Length: 142
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: ABUBAKARI-SADIQUE HAMIDU</p><p>WSD-11392</p></body></html>
* Connection #0 to host 127.0.0.1:80 left intact

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
#
```

Step 6 Keeping the Evidence Safe

I made a copy of the recording to actually work on, and left the original completely untouched. Then I ran a command that produces a kind of digital fingerprint of a file (this is called a hash). I made a fingerprint of both the original and the copy, and they came out exactly the same proof that the copy is identical to the original and nothing was changed.

ad930f9ad74e0b2da5d726d51dff79c5cb77834c7e29cb3f1897925a3dfe99cc

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker run -it --name sbtdf203-lab --cap...

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab1]
# tshark -i lo -f 'tcp port 80' -w evidence/basic.pcapng
Running as user "root" and group "root". This could be dangerous.
Capturing on 'Loopback: lo'
10 ^C

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab1]
# tshark -r evidence/basic.pcapng | wc -l
Running as user "root" and group "root". This could be dangerous.
10

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab1]
# ls -lh evidence/basic.pcapng
-rw-r--r-- 1 root root 1.9K Sep  8 16:12 evidence/basic.pcapng

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab1]
# tshark -r evidence/basic.pcapng | wc -l
Running as user "root" and group "root". This could be dangerous.
10

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab1]
#

sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker exec -it sbt...

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab1]
# curl --no-keepalive -v http://127.0.0.1/basic.html
* Trying 127.0.0.1:80...
* Established connection to 127.0.0.1 (127.0.0.1 port 80) from 127.0.0.1
* using HTTP/1.x
> GET /basic.html HTTP/1.1
> Host: 127.0.0.1
> User-Agent: curl/8.21.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 08 Sep 2026 16:10:30 GMT
< Server: Apache/2.4.68 (Debian)
< Last-Modified: Tue, 08 Sep 2026 15:21:57 GMT
< ETag: "8e-65afa4ba94f11"
< Accept-Ranges: bytes
< Content-Length: 142
< Vary: Accept-Encoding
< Content-Type: text/html
<
<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: ABUBAKARI-SADIQUE
WSD-11392</p></body></html>
* Connection #0 to host 127.0.0.1:80 left intact

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab1]
#
```

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker run -it --name sbtdf203-lab --cap-add=NET_RAW --cap-add=NET_ADMIN -p 8080:80 kaliinu... b1 — docker exec -it sbt...

MAY22

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab1]
# cp --preserve=timestamps evidence/basic.pcapng working/basic_working.pcapng

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab1]
# sha256sum evidence/basic.pcapng working/basic_working.pcapng | tee reports/capture_hashes.txt
ad930f9ad74e0b2da5d726d51dff79c5cb77834c7e29cb3f1897925a3dfe99cc evidence/basic.pcapng
ad930f9ad74e0b2da5d726d51dff79c5cb77834c7e29cb3f1897925a3dfe99cc working/basic_working.pcapng

(root@ 1f1f5ef2d179)-[~/SBT-DF203-Lab1]
#

sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker exec -it sbt...

MAY22

root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker exec -it sbt...
#
```

Step 7 How the Connection Opened

Every connection like this starts with a short back-and-forth to open the line, sometimes called a "handshake." Here's what it looked like, in plain terms:

Message	Who sent it	What it means, in plain words
1	My computer → the webpage	"I'd like to open a connection with you."
2	The webpage → my computer	"Okay, let's connect here's proof I'm listening."
3	My computer → the webpage	"Got it. We're connected."

Each side also keeps a running count of how many pieces of information it has sent, a bit like a page counter. That counter is what lets both sides check that nothing got lost or duplicated along the way. All three of these opening messages happened within a tiny fraction of a second the entire handshake was done in about 0.00004 seconds.

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker run -it --name sbtdf203-lab --cap-add=NET_RAW --cap-add=NET_ADMIN -p 8080:80 kaliinu...
(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# cp --preserve=timestamps evidence/basic.pcapng working/basic_working.pcapng

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# sha256sum evidence/basic.pcapng working/basic_working.pcapng | tee reports/capture_hashes.txt
ad930f9ad74e0b2da5d726d51dff79c5cb77834c7e29cb3f1897925a3dfe99cc evidence/basic.pcapng
ad930f9ad74e0b2da5d726d51dff79c5cb77834c7e29cb3f1897925a3dfe99cc working/basic_working.pcapng

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# tshark -r working/basic_working.pcapng -Y 'tcp.stream eq 0 && tcp.flags.syn==1' \
Running as user "root" and group "root". This could be dangerous.
  1 0.000000000 127.0.0.1 → 127.0.0.1 TCP 74 45318 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=1258369171 TSecr=0 WS=128
  2 0.000020775 127.0.0.1 → 127.0.0.1 TCP 74 80 → 45318 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=1258369171 TSecr=1258369171 WS=128

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# -T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport \
bash: -T: command not found

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# tshark -r working/basic_working.pcapng -Y 'tcp.stream eq 0 && tcp.flags.syn==1' -T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags -e tcp.seq -e tcp.ack | tee reports/handshake.txt
Running as user "root" and group "root". This could be dangerous.
  1 2026-09-08T16:10:30.532287365+0000 127.0.0.1 45318 127.0.0.1 80 0x0002 0 0
  2 2026-09-08T16:10:30.532308140+0000 127.0.0.1 80 127.0.0.1 45318 0x0012 0 1

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
#
```

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker run -it --name sbtdf203-lab --cap-add=NET_RAW --cap-add=NET_ADMIN -p 8080:80 kaliinu...
(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# tshark -r working/basic_working.pcapng
Running as user "root" and group "root". This could be dangerous.
  1 0.000000000 127.0.0.1 → 127.0.0.1 TCP 74 45318 → 80 [SYN] Seq=0 Win=65495 Len=0 MSS=65495 SACK_PERM TSval=1258369171 TSecr=0 WS=128
  2 0.000020775 127.0.0.1 → 127.0.0.1 TCP 74 80 → 45318 [SYN, ACK] Seq=0 Ack=1 Win=65483 Len=0 MSS=65495 SACK_PERM TSval=1258369171 TSecr=1258369171 WS=128
  3 0.000040708 127.0.0.1 → 127.0.0.1 TCP 66 45318 → 80 [ACK] Seq=1 Ack=1 Win=65536 Len=0 TSval=1258369171 TSecr=1258369171
  4 0.000316359 127.0.0.1 → 127.0.0.1 HTTP 149 GET /basic.html HTTP/1.1
  5 0.000398547 127.0.0.1 → 127.0.0.1 TCP 66 80 → 45318 [ACK] Seq=1 Ack=84 Win=65408 Len=0 TSval=1258369172 TSecr=1258369172
  6 0.002073377 127.0.0.1 → 127.0.0.1 HTTP 459 HTTP/1.1 200 OK (text/html)
  7 0.002275730 127.0.0.1 → 127.0.0.1 TCP 66 45318 → 80 [ACK] Seq=84 Ack=394 Win=65152 Len=0 TSval=1258369173 TSecr=1258369173
  8 0.002737229 127.0.0.1 → 127.0.0.1 TCP 66 45318 → 80 [FIN, ACK] Seq=84 Ack=394 Win=65536 Len=0 TSval=1258369174 TSecr=1258369173
  9 0.007706181 127.0.0.1 → 127.0.0.1 TCP 66 80 → 45318 [FIN, ACK] Seq=394 Ack=85 Win=65536 Len=0 TSval=1258369179 TSecr=1258369174
 10 0.007755968 127.0.0.1 → 127.0.0.1 TCP 66 45318 → 80 [ACK] Seq=85 Ack=395 Win=65536 Len=0 TSval=1258369179 TSecr=1258369179

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
#
```

Step 8 The Actual Question and Answer

Once the connection was open, my computer sent a proper request: "GET /basic.html" plain English for "please send me the file called basic.html." Apache replied straight away with "200 OK," followed by the actual page content: my name and student ID, sent back as plain, readable text.

Putting the request and the reply together in one block looks like this:

```
GET /basic.html HTTP/1.1
Host: 127.0.0.1
User-Agent: curl/8.21.0

HTTP/1.1 200 OK
Server: Apache/2.4.68 (Debian)
Content-Type: text/html

...Name: Abubakari-Sadique Hamidu...Reg No: 2025-FWSD-11392...
```

The important point here: none of this was scrambled or hidden in any way. Anyone watching this connection a stranger on the same network, for example could read the exact same request and reply I just did, with nothing to decode. That's not a mistake on my part; that's simply what an ordinary, non-secure web connection looks like. It's exactly why real websites that handle anything sensitive use a secure version instead.


```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker run -it --name sbtdf203-lab --cap-add=NET_RAW --cap-add=NET_ADMIN -p 8080:80 kaliinu...
(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# tshark -r working/basic_working.pcapng -Y 'http.request' -T fields -e frame.number -e frame.time -e ip.src -e tcp.sr
.srcport -e ip.dst -e tcp.dstport -e http.request.method -e http.request.uri -e http.host -e http.user_agent | tee re
ports/http_requests.tsv
Running as user "root" and group "root". This could be dangerous.
4      2026-09-08T16:10:30.532603724+0000      127.0.0.1      45318      127.0.0.1      80      GET      /basic.html 1
27.0.0.1      curl/8.21.0

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# tshark -r working/basic_working.pcapng -Y 'http.response' -T fields -e frame.number -e frame.time -e http.respons
e.code -e http.response.phrase -e http.server -e http.content_type -e http.content_length | tee reports/http_response
s.tsv
Running as user "root" and group "root". This could be dangerous.
6      2026-09-08T16:10:30.534360742+0000      200      OK      Apache/2.4.68 (Debian)      text/html      142

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
#
```

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker run -it --name sbtdf203-lab --cap-add=NET_RAW --cap-add=NET_ADMIN -p 8080:80 kaliinu...
(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# tshark -r working/basic_working.pcapng -q -z follow,tcp,ascii,0 | tee reports/follow_stream.txt
Running as user "root" and group "root". This could be dangerous.

=====
Follow: tcp,ascii
Filter: tcp.stream eq 0
Node 0: 127.0.0.1:45318
Node 1: 127.0.0.1:80
83
GET /basic.html HTTP/1.1
Host: 127.0.0.1
User-Agent: curl/8.21.0
Accept: */*

393
HTTP/1.1 200 OK
Date: Tue, 08 Sep 2026 16:10:30 GMT
Server: Apache/2.4.68 (Debian)
Last-Modified: Tue, 08 Sep 2026 15:21:57 GMT
ETag: "8e-65afa4ba94f11"
Accept-Ranges: bytes
Content-Length: 142
Vary: Accept-Encoding
Content-Type: text/html

<!DOCTYPE html>
<html><body><h1>SBT-DF203 HTTP Evidence</h1><p>Name: ABUBAKARI-SADIQUE HAMIDU</p><p>Reg No: 2025-FWSD-11392</p></body>
</html>

=====
(root@1f1f5ef2d179)~/SBT-DF203-Lab1
#
```


Step 9 How the Message Was Wrapped

A single piece of network traffic isn't just "the message" on its own it's wrapped in layers, a bit like a letter inside an envelope inside a parcel. I looked closely at one piece of traffic (my request for the page) and pulled the layers apart to see what was inside each one:

Layer (outside to inside)	Size	What it's for
Outer wrapper	14 bytes	Basic delivery info for this one hop
Addressing layer	20 bytes	Says where the message is going and coming from
Conversation-tracking layer	32 bytes	Keeps the two sides in sync, counts what's been sent
The actual message	83 bytes	The plain-text request itself: "GET /basic.html..."

Add all of those up and they match the total size of the piece of traffic exactly (149 bytes) nothing missing, nothing extra. That's really all "the message is wrapped in layers" means once you see the actual numbers.

One genuinely interesting thing turned up in that outer wrapper: normally, every device on a network has its own hardware address baked into its network card a bit like a serial number, usually called a MAC address. Here, both the "from" and "to" hardware addresses showed up as all zeros. That's because this whole test never left my laptop: my computer was talking to itself, so there was no real network card involved on either end, and nothing to put a genuine hardware address on. A real hardware address only matters once a message actually has to travel across a wire or through a switch to a different device which didn't happen here.

```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker run -it --name sbtdf203-lab --cap-add=NET_RAW --cap-add=NET_ADMIN -p 8080:80 kaliinu...

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
# tshark -r working/basic_working.pcapng -V -Y 'frame.number==4' | tee reports/encapsulation_frame4.txt
Running as user "root" and group "root". This could be dangerous.
Frame 4: Packet, 149 bytes on wire (1192 bits), 149 bytes captured (1192 bits) on interface lo, id 0
  Section number: 1
    Interface id: 0 (lo)
      Interface name: lo
      Interface description: Loopback
      Encapsulation type: Ethernet (1)
      Arrival Time: Sep  8, 2026 16:10:30.532603724 UTC
      UTC Arrival Time: Sep  8, 2026 16:10:30.532603724 UTC
      Epoch Arrival Time: 1788883830.532603724
      [Time shift for this packet: 0.000000000 seconds]
      [Time delta from previous captured frame: 275.651 microseconds]
      [Time since reference or first frame: 316.359 microseconds]
      Frame Number: 4
      Frame Length: 149 bytes (1192 bits)
      Capture Length: 149 bytes (1192 bits)
      [Frame is marked: False]
      [Frame is ignored: False]
      [Protocols in frame: eth:ethertype:ip:tcp:http]
      Character encoding: ASCII (0)
      Ethernet II, Src: 00:00:00_00:00:00 (00:00:00:00:00:00), Dst: 00:00:00_00:00:00 (00:00:00:00:00:00)
        Destination: 00:00:00_00:00:00 (00:00:00:00:00:00)
          .... 0. .... = LG bit: Globally unique address (factory default)
          .... 0. .... = IG bit: Individual address (unicast)
        Source: 00:00:00_00:00:00 (00:00:00:00:00:00)
          .... 0. .... = LG bit: Globally unique address (factory default)
          .... 0. .... = IG bit: Individual address (unicast)
        Type: IPv4 (0x0800)
        [Stream index: 0]
      Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
        0100 .... = Version: 4
        .... 0101 = Header Length: 20 bytes (5)
```

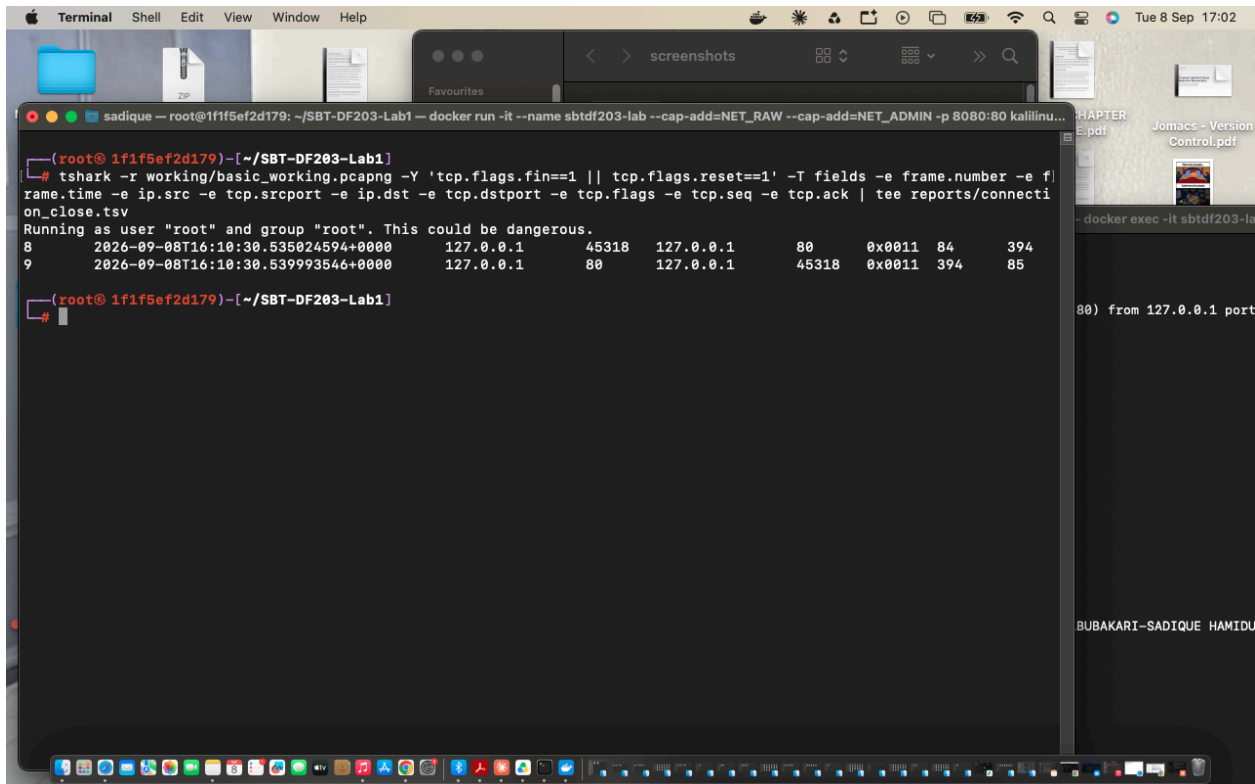
```
Terminal Shell Edit View Window Help
sadique — root@1f1f5ef2d179: ~/SBT-DF203-Lab1 — docker run -it --name sbtdf203-lab --cap-add=NET_RAW --cap-add=NET_ADMIN -p 8080:80 kaliinu...

[Checksum Status: Unverified]
Urgent Pointer: 0
Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
  TCP Option - No-Operation (NOP)
    Kind: No-Operation (1)
  TCP Option - No-Operation (NOP)
    Kind: No-Operation (1)
  TCP Option - Timestamps: TSval 1258369172, TSecr 1258369171
    Kind: Time Stamp Option (8)
    Length: 10
    Timestamp value: 1258369172
    Timestamp echo reply: 1258369171
[Timestamps]
  [Time since first frame in this TCP stream: 316.359 microseconds]
  [Time since previous frame in this TCP stream: 275.651 microseconds]
[SEQ/ACK analysis]
  [iRTT: 40.708 microseconds]
  [Bytes in flight: 83]
  [Bytes sent since last PSH flag: 83]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]
TCP payload (83 bytes)
Hypertext Transfer Protocol
GET /basic.html HTTP/1.1\r\n
  Request Method: GET
  Request URI: /basic.html
  Request Version: HTTP/1.1
Host: 127.0.0.1\r\n
User-Agent: curl/8.21.0\r\n
Accept: */*\r\n
\r\n
[Full request URI: http://127.0.0.1/basic.html]

(root@1f1f5ef2d179)~/SBT-DF203-Lab1
```

Step 10 How the Connection Closed

Once there was nothing left to send, my computer said "I'm done sending," the webpage said the same thing back, and my computer gave one last "got it" to finish. Three short messages, and the connection closed cleanly no errors, nothing cut off or forced shut.



```
(root@1f1f5ef2d179) [~/SBT-DF203-Lab1]
# tshark -r working/basic_working.pcapng -Y 'tcp.flags.fin==1 || tcp.flags.reset==1' -T fields -e frame.number -e frame.time -e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.flags -e tcp.seq -e tcp.ack | tee reports/connection_close.tsv
Running as user "root" and group "root". This could be dangerous.
8      2026-09-08T16:10:30.535024594+0000      127.0.0.1      45318      127.0.0.1      80      0x0011      84      394
9      2026-09-08T16:10:30.539993546+0000      127.0.0.1      80      127.0.0.1      45318      0x0011      394      85

(root@1f1f5ef2d179) [~/SBT-DF203-Lab1]
#
```

What I Found Summary

Question	Answer
How long did the whole thing take?	About 0.0078 seconds, start to finish
Who was talking to whom?	My own computer, talking to itself (127.0.0.1), on ports 45318 and 80
What page was asked for?	/basic.html
What answer came back?	200 OK success with my name and ID as plain text
How did it end?	Cleanly, both sides agreeing to close, no errors
Were real hardware addresses visible?	No both showed as all zeros, because it never left my own computer

What This Lab Taught Me

The main thing I take away from this is how much genuinely useful information sits in ordinary network traffic that most people never look at. From this one tiny recording ten small pieces of data, done in under a hundredth of a second I could work out exactly who talked to whom, what time it happened, what was asked for, and what was sent back, without needing to unscramble or decode anything, because none of it was encrypted in the first place.

That's really the core of what a network forensic analyst does: read that plain information carefully, piece by piece, and be able to explain exactly what each piece proves.

Appendix Commands I Used

Command	What it does, in plain words
<code>curl -v URL</code>	Ask a webpage for its content, and show the full reply
<code>tshark -i lo -w file.pcapng</code>	Record all network traffic on this computer to a file
<code>tshark -r file -Y 'http.request'</code>	Pull out just the request from a recording
<code>tshark -r file -Y 'http.response'</code>	Pull out just the reply from a recording
<code>tshark -r file -z follow,tcp,ascii,0</code>	Put the whole conversation back together, readable
<code>tshark -r file -V -Y 'frame.number==N'</code>	Show every layer of one specific piece of traffic
<code>sha256sum file</code>	Make a digital fingerprint of a file, to prove it hasn't changed

All 12 numbered screenshots referenced above, plus a full set of 21 screenshots in the order I actually took them, are included in the screenshots folder of the submitted project.

Student Declaration

I confirm that I did this lab myself, in a safe and isolated setup, kept the original recording untouched, never touched anyone else's computer or network, and that everything written above is my own work.

Signed: Abubakari-Sadique Hamidu

Student ID: 2025-FWSD-11392

Date: 8 September 2026